

**ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA TOÁN - CƠ - TIN HỌC**



BÁO CÁO KẾT THÚC THỰC TẬP

**Ứng dụng WebSocket trong việc xây dựng trang web xem
video đồng bộ trạng thái độ trễ thấp**

Họ và tên sinh viên: Tạ Tiến Nam

Mã sinh viên: 23000143

Lớp: K68A2 Toán Tin

Ngành: Toán Tin

Khoa: Toán - Cơ - Tin học

Trường: Đại học Khoa học Tự nhiên (VNU - HUS)

Hà Nội, Năm 2026

LỜI CẢM ƠN

Lời đầu tiên, em xin gửi lời cảm ơn chân thành và sâu sắc nhất tới toàn thể các thầy cô giáo trong Khoa Toán - Cơ - Tin học, Trường Đại học Khoa học Tự nhiên – Đại học Quốc gia Hà Nội đã tận tình giảng dạy, truyền đạt những kiến thức chuyên môn vững chắc và tư duy logic quý báu cho em trong suốt quá trình học tập tại trường.

Đặc biệt, em xin bày tỏ lòng biết ơn sâu sắc tới các thầy cô hướng dẫn môn học Thực tập chuyên ngành đã luôn theo sát, định hướng chuyên môn quý báu giúp em hoàn thiện đề tài **”Ứng dụng WebSocket trong việc xây dựng trang web xem video đồng bộ trạng thái độ trễ thấp”**.

Dù đã cố gắng hoàn thiện báo cáo và ứng dụng một cách cẩn chu, bài báo cáo không tránh khỏi những hạn chế nhất định. Em rất mong nhận được sự đóng góp ý kiến từ phía thầy cô để tiếp tục phát triển sản phẩm tốt hơn.

Em xin chân thành cảm ơn!

Hà Nội, ngày 15 tháng 08 năm 2026

Sinh viên thực hiện

Tạ Tiến Nam

DANH MỤC TỪ VIẾT TẮT

Từ viết tắt	Tên tiếng Anh đầy đủ	Ý nghĩa tiếng Việt
API	Application Programming Interface	Giao diện lập trình ứng dụng
CORS	Cross-Origin Resource Sharing	Chia sẻ tài nguyên khác nguồn
HTTP / HTTPS	Hypertext Transfer Protocol (Secure)	Giao thức truyền tải siêu văn bản (Bảo mật)
JSON	JavaScript Object Notation	Định dạng trao đổi dữ liệu JSON
JWT	JSON Web Token	Mã thông báo xác thực chuỗi mã hóa
REST	Representational State Transfer	Kiểu kiến trúc truyền tải trạng thái đại diện
SPA	Single Page Application	Ứng dụng trang đơn
WebSocket	WebSocket Protocol	Giao thức truyền thông 2 chiều qua TCP
RDBMS	Relational Database Management System	Hệ quản trị cơ sở dữ liệu quan hệ

Danh sách bảng

1	Cấu trúc các bảng cơ sở dữ liệu PostgreSQL của hệ thống	8
2	Bảng tổng hợp kết quả kiểm thử các chức năng chính	11
3	Đánh giá độ trễ đồng bộ video thời gian thực trong các kịch bản mạng	12

Danh sách hình vẽ

1	Sơ đồ thực thể liên kết (ERD) Cơ sở dữ liệu PostgreSQL	8
2	Sơ đồ luồng xử lý đồng bộ trạng thái phát video thời gian thực	9

Mục lục

LỜI CẢM ƠN	1
DANH MỤC TỪ VIẾT TẮT	2
DANH MỤC BẢNG BIỂU	3
DANH MỤC HÌNH ẢNH, SƠ ĐỒ	4
1 PHẦN 1: GIỚI THIỆU	6
1.1 Phát biểu bài toán và công việc được giao	6
1.2 Khảo sát công nghệ, công cụ và lý do lựa chọn	6
1.2.1 Giới thiệu công nghệ WebSocket	6
1.2.2 Giới thiệu công nghệ xác thực: JWT và Bcrypt	7
1.2.3 Backend & Cơ sở dữ liệu: Node.js, PostgreSQL và Redis	7
2 PHẦN 2: CÔNG VIỆC TRIỂN KHAI	8
2.1 Phân tích yêu cầu và Thiết kế kiến trúc hệ thống	8
2.1.1 Thiết kế Cơ sở dữ liệu PostgreSQL	8
2.1.2 Sơ đồ luồng dữ liệu	9
2.2 Triển khai các Module chức năng	9
2.2.1 Module Xác thực và Quản lý phòng xem	9
2.2.2 Module Đồng bộ Video và Thuật toán Bù sai lệch	9
2.2.3 Module Trò chuyện nhóm thời gian thực	10
3 PHẦN 3: CÁC KẾT QUẢ ĐẠT ĐƯỢC	11
3.1 Giới thiệu sản phẩm phần mềm và Giao diện	11
3.2 Kết quả kiểm thử và Phân tích lỗi kỹ thuật	11
3.2.1 Kết quả các Test Cases kiểm thử chức năng	11
3.2.2 Phân tích lỗi kỹ thuật và Cách khắc phục	11
3.2.3 Đánh giá độ trễ đồng bộ (Latency Benchmark)	12
3.3 Đánh giá ưu nhược điểm & Bài học kinh nghiệm	12
KẾT LUẬN	13
TÀI LIỆU THAM KHẢO	14
PHỤ LỤC	15

1 PHẦN 1: GIỚI THIỆU

1.1 Phát biểu bài toán và công việc được giao

Nhu cầu xem video cùng nhau giữa những người dùng ở các vị trí địa lý khác nhau (Co-watching / Xem video nhóm) ngày càng phổ biến. Tuy nhiên, các nền tảng video hiện tại chủ yếu phục vụ cá nhân hóa theo từng phiên đơn lẻ. Việc đồng bộ phát video thủ công (đếm ngược 3-2-1 để bấm Play) gặp hạn chế lớn về sai lệch độ trễ mạng ($2s - 10s$) và làm mất tính tương tác thời gian thực khi có người tạm dừng video để thảo luận.

Công việc được giao trong đợt thực tập: Nhiệm vụ trọng tâm là nghiên cứu và xây dựng ứng dụng Web thời gian thực cho phép đồng bộ hóa trạng thái phát video nhóm với độ trễ siêu thấp ($< 1s$). Các mục tiêu cụ thể gồm:

1. Đăng ký, đăng nhập tài khoản an toàn với mã hóa mật khẩu và token xác thực.
2. Khởi tạo phòng xem và tham gia phòng thông qua Mã phòng (Room ID) duy nhất.
3. Đồng bộ tức thời các thao tác: Phát (Play), Tạm dừng (Pause), Tua (Seek), Chuyển bài trong danh sách phát (Queue).
4. Tích hợp hệ thống trò chuyện (Chat) gắn mốc thời gian xem video (Timestamp).

1.2 Khảo sát công nghệ, công cụ và lý do lựa chọn

1.2.1 Giới thiệu công nghệ WebSocket

WebSocket là một giao thức truyền thông mạng hoạt động trên nền TCP, được chuẩn hóa bởi IETF trong RFC 6455, cho phép thiết lập một kênh truyền thông **song công toàn phần (full-duplex)** giữa Client và Server trên một kết nối duy nhất. Khác với mô hình HTTP truyền thống vốn chỉ cho phép Client chủ động gửi yêu cầu và Server phản hồi (request-response), WebSocket duy trì kết nối mở liên tục, cho phép cả hai phía chủ động gửi dữ liệu tới nhau bất kỳ lúc nào mà không cần thiết lập lại kết nối.

Quá trình thiết lập kết nối WebSocket bắt đầu bằng một *HTTP Handshake* (yêu cầu nâng cấp giao thức thông qua header Upgrade: websocket), sau đó kết nối được chuyển đổi hoàn toàn sang giao thức WebSocket. Từ thời điểm này, dữ liệu được truyền qua các *frame* nhị phân có kích thước tiêu đề rất nhỏ (chỉ $2B - 10B$), giúp giảm đáng kể độ trễ và chi phí băng thông so với việc gửi lặp lại các HTTP Header đầy đủ trong mỗi request.

Trong đề tài này, thư viện **Socket.IO** được lựa chọn làm lớp trừu tượng hóa trên nền WebSocket, do cung cấp sẵn các tính năng thiết yếu cho ứng dụng thời gian thực:

- **Cơ chế Room:** Cho phép nhóm các kết nối (socket) theo từng phòng xem, giúp việc phát sự kiện (broadcast) chỉ giới hạn đúng phạm vi người dùng liên quan.

- **Tự động kết nối lại (Auto-reconnect):** Khi mạng gián đoạn tạm thời, client tự động thiết lập lại kết nối mà không cần người dùng thao tác thủ công.
- **Cơ chế dự phòng (Fallback):** Tự động chuyển sang HTTP long-polling trong trường hợp trình duyệt hoặc mạng không hỗ trợ WebSocket, đảm bảo tính tương thích rộng rãi.

Nhờ các đặc tính trên, WebSocket là nền tảng phù hợp để xây dựng module đồng bộ hóa trạng thái phát video thời gian thực với độ trễ siêu thấp ($< 300ms$), đáp ứng đúng mục tiêu đề tài đặt ra.

1.2.2 Giới thiệu công nghệ xác thực: JWT và Bcrypt

Hệ thống xác thực người dùng được xây dựng dựa trên hai công nghệ chính: **JWT (JSON Web Token)** cho việc cấp phát và xác minh danh tính, và **Bcrypt** cho việc băm mật khẩu.

JWT là một chuẩn mở (RFC 7519) cho phép tạo ra chuỗi mã thông báo mang thông tin xác thực đã được ký số bằng thuật toán HMAC-SHA256. Do bản chất *stateless* (không lưu trạng thái phiên tại Server), JWT rất phù hợp với kiến trúc ứng dụng thời gian thực: token được đính kèm dễ dàng trong REST Header (`Authorization: Bearer <token>`) lẫn trong WebSocket Handshake (`socket.handshake.auth.token`), giúp thống nhất cơ chế xác thực giữa hai luồng giao tiếp REST API và WebSocket.

Bcrypt là thuật toán băm mật khẩu một chiều có tích hợp *salt* ngẫu nhiên và hệ số chi phí tính toán (*cost factor*) có thể điều chỉnh, giúp tăng thời gian cần thiết cho các cuộc tấn công vét cạn (brute-force) hoặc tấn công bảng cầu vồng (rainbow table), qua đó bảo vệ mật khẩu người dùng ngay cả khi cơ sở dữ liệu bị rò rỉ.

1.2.3 Backend & Cơ sở dữ liệu: Node.js, PostgreSQL và Redis

- **Node.js & TypeScript:** Event Loop bất đồng bộ (Non-blocking I/O) xử lý hàng ngàn kết nối WebSocket đồng thời với bộ nhớ RAM thấp.
- **PostgreSQL & Redis (ioredis):** PostgreSQL lưu trữ bền vững thông tin tài khoản, phòng xem và lịch sử tin nhắn. Redis lưu trữ In-memory siêu tốc ($< 1ms$) cho trạng thái phát thời gian thực (`isPlaying`, `progress`, `lastUpdated`).

2 PHẦN 2: CÔNG VIỆC TRIỂN KHAI

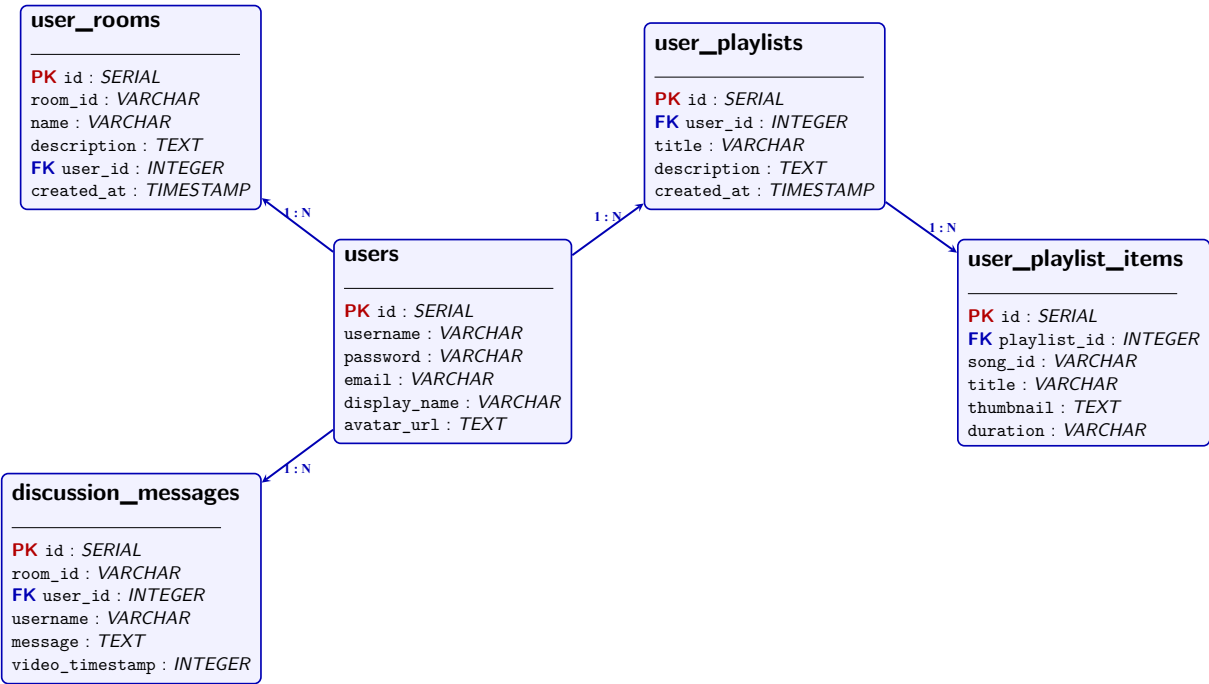
2.1 Phân tích yêu cầu và Thiết kế kiến trúc hệ thống

2.1.1 Thiết kế Cơ sở dữ liệu PostgreSQL

Hệ thống sử dụng 5 bảng dữ liệu quan hệ được chuẩn hóa:

Bảng 1: Cấu trúc các bảng cơ sở dữ liệu PostgreSQL của hệ thống

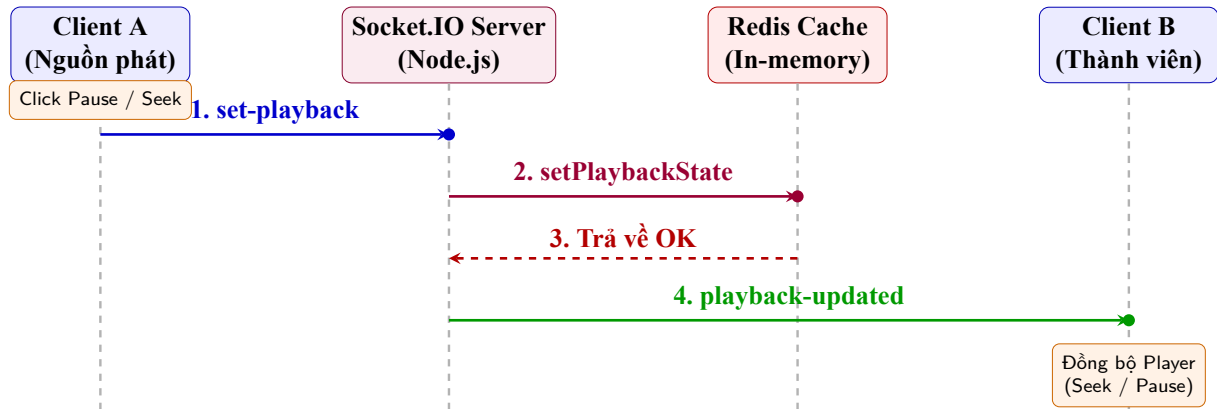
Tên bảng	Trường thông tin	Mô tả & Khóa
users	id, username, password, email, display_name, avatar_url	PK: id, UNIQUE: username, email. Lưu tài khoản người dùng.
user_rooms	id, room_id, name, description, user_id, created_at	PK: id, FK: user_id → users(id). Quản lý thông tin phòng.
user_playlists	id, user_id, title, description, created_at	PK: id, FK: user_id → users(id). Lưu danh sách phát cá nhân.
user_playlist_items	id, playlist_id, song_id, title, thumbnail, duration	PK: id, FK: playlist_id → user_playlists(id). Chi tiết bài hát.
discussion_messages	id, room_id, user_id, username, message, video_timestamp	PK: id, FK: user_id → users(id). Lưu lịch sử chat phòng.



Hình 1: Sơ đồ thực thể liên kết (ERD) Cơ sở dữ liệu PostgreSQL

2.1.2 Sơ đồ luồng dữ liệu

- **Luồng đăng nhập JWT:** Client gửi thông tin đăng nhập → Server xác thực Bcrypt hash → Tạo chuỗi JWT mã hóa trả về cho Client lưu vào LocalStorage.
- **Luồng đồng bộ trạng thái phát video (Sequence Flow):**



Hình 2: Sơ đồ luồng xử lý đồng bộ trạng thái phát video thời gian thực

2.2 Triển khai các Module chức năng

2.2.1 Module Xác thực và Quản lý phòng xem

- **REST API Auth:** Xây dựng các route đăng ký, đăng nhập và middleware `authenticateToken` giải mã JWT Token trong Header `Authorization: Bearer <token>`.
- **WebSocket Handshake Auth:** Khi thiết lập kết nối socket, Server lấy token từ `socket.handshake.auth.token` để xác thực danh tính người dùng và phân nhóm socket vào Room tương ứng qua lệnh `socket.join(roomId)`.

2.2.2 Module Đồng bộ Video và Thuật toán Bù sai lệch

- **Khắc phục vòng lặp sự kiện (Event Loop Feedback Loop):** Khi Client nhận lệnh đồng bộ từ Server và gọi API trình phát (`seekTo`, `pauseVideo`), sự kiện `onStateChange` tự động bật làm gửi ngược sự kiện lên Server. Hệ thống sử dụng cờ `isSyncing = true` tại Client để tạm thời bỏ qua phát sự kiện trong `300ms`.
- **Thuật toán bù sai lệch thời gian (Drift Compensation):** Khi người dùng mới tham gia phòng, Server tính thời gian video đã phát (tính bằng giây) bằng công thức, trong đó `lastUpdated` là mốc thời gian (Unix timestamp, đơn vị mili-giây) được lưu trong Redis

tại lần cập nhật gần nhất, và phép chia cho 1000 dùng để quy đổi từ mili-giây sang giây:

$$\text{progress (s)} = \text{progress_gốc (s)} + \frac{\text{Thời gian hiện tại (ms)} - \text{lastUpdated (ms)}}{1000}$$

2.2.3 Module Trò chuyện nhóm thời gian thực

Server lắng nghe sự kiện `send-discussion-message`, trích xuất `videoTimestamp` hiện tại, lưu tin nhắn vào PostgreSQL và phát sự kiện `new-discussion-message` cho toàn bộ thành viên trong phòng.

3 PHẦN 3: CÁC KẾT QUẢ ĐẠT ĐƯỢC

3.1 Giới thiệu sản phẩm phần mềm và Giao diện

Hệ thống đã hoàn thiện 100% các chức năng với giao diện Web Single Page Application (SPA) responsive:

1. **Giao diện Đăng nhập / Đăng ký:** Xác thực dữ liệu đầu vào và lưu trữ Token an toàn.
2. **Giao diện Quản lý phòng xem:** Tạo phòng mới và truy cập nhanh bằng Mã phòng (Room ID).
3. **Giao diện Phòng xem trực tuyến:** Trình phát video HTML5 / YouTube, Khung danh sách phát bài hát (Queue), Khung trò chuyện nhóm đính kèm Timestamp và Danh sách thành viên online.

3.2 Kết quả kiểm thử và Phân tích lỗi kỹ thuật

3.2.1 Kết quả các Test Cases kiểm thử chức năng

Bảng 2: Bảng tổng hợp kết quả kiểm thử các chức năng chính

STT	Kịch bản kiểm thử (Test Case)	Kết quả kỳ vọng	Trạng thái
1	Đăng ký & Đăng nhập tài khoản	Tạo chuỗi Token JWT, lưu vào LocalStorage	PASSED
2	Tạo phòng mới & Nhận Room ID	Khởi tạo bản ghi phòng mới trong PostgreSQL	PASSED
3	Kết nối WebSocket Handshake	Xác thực Token JWT, tự động join Room	PASSED
4	Thêm video YouTube vào Queue	Đồng bộ danh sách phát tới tất cả thành viên	PASSED
5	Đồng bộ sự kiện Play / Pause / Seek	Các Client đồng bộ trạng thái phát trong $< 300ms$	PASSED
6	Chat tin nhắn đính kèm Timestamp	Tin nhắn hiển thị link thời gian nhảy video	PASSED

3.2.2 Phân tích lỗi kỹ thuật và Cách khắc phục

1. **Lỗi vòng lặp sự kiện vô hạn:** Gọi lệnh API trình phát kích hoạt lại sự kiện `onStateChange`. *Khắc phục:* Sử dụng cờ cần `isSyncing` tạm khóa gửi tin nhắn lên Server trong $300ms$.

2. **Lỗi trôi thời gian phát (Time Drift):** Thời gian phát cố định khiến người vào sau bị lệch pha. *Khắc phục:* Thuật toán Drift Compensation tự động tính thời gian trôi qua dựa trên `lastUpdated` lưu trong Redis.
3. **Lỗi rò rỉ tệp tin rác trên đĩa:** Video tự tải lên không bị xóa khi bỏ khỏi Queue. *Khắc phục:* Xây dựng hàm `cleanupCustomVideoFile()` xóa tệp tin bằng `fs.unlinkSync()` ngay khi gỡ bài.
4. **Lỗi Token JWT hết hạn:** Ngắt kết nối WebSocket giữa chừng. *Khắc phục:* Đăng ký handler `auth-error` hiển thị thông báo và chuyển hướng đăng nhập lại.

3.2.3 Đánh giá độ trễ đồng bộ (Latency Benchmark)

Độ trễ đồng bộ được đo bằng cách ghi nhận khoảng thời gian từ lúc Client A phát sự kiện điều khiển (`set-playback`) tới lúc Client B nhận được sự kiện `playback-updated` tương ứng, sử dụng đồng hồ `performance.now()` tại cả hai đầu và đồng bộ lịch giờ hệ thống trước khi đo. Mỗi kịch bản mạng được đo lặp lại 50 lần liên tiếp để lấy giá trị tối thiểu, trung bình và tối đa:

Bảng 3: Đánh giá độ trễ đồng bộ video thời gian thực trong các kịch bản mạng

Môi trường mạng kiểm thử	Độ trễ tối thiểu	Độ trễ trung bình	Độ trễ tối đa
Mạng nội bộ (LAN / Local-host)	12ms	25ms	45ms
Wifi cáp quang (Hà Nội)	45ms	85ms	150ms
Mạng di động 4G / Ngrok Tunnel	120ms	210ms	380ms

3.3 Đánh giá ưu nhược điểm & Bài học kinh nghiệm

- **Ưu điểm:** Độ trễ siêu thấp ($< 300ms$), xác thực JWT bảo mật, xử lý triệt để các sự cố vòng lặp và dọn dẹp bộ nhớ.
- **Hạn chế:** Khi kết nối mạng của Client bị chập chờn quá mức (Packet loss), trình phát bị nạp đệm (Buffering) gây lệch nhẹ trước khi đợt Sync tiếp theo cân chỉnh.
- **Bài học kinh nghiệm:** Nâng cao tư duy thiết kế hệ thống thời gian thực bất đồng bộ, kết hợp PostgreSQL & Redis và kỹ năng phân tích nguyên nhân lỗi kỹ thuật.

KẾT LUẬN

Báo cáo đã hoàn thành đầy đủ các yêu cầu chuyên môn. Sản phẩm giải quyết bài toán đồng bộ phiên xem video nhóm qua mạng với độ trễ thấp, đem lại trải nghiệm tương tác mượt mà. Thông qua đợt thực tập, sinh viên đã củng cố kiến thức đào tạo tại Khoa Toán - Cơ - Tin học và làm chủ công nghệ lập trình Web thời gian thực.

Trong thời gian tới, đề tài có thể được mở rộng theo một số hướng: bổ sung cơ chế phân quyền chủ phòng (host) để kiểm soát quyền điều khiển video; áp dụng Redis Adapter cho Socket.IO nhằm mở rộng hệ thống theo chiều ngang (horizontal scaling) khi số lượng người dùng tăng cao; tích hợp thêm các nguồn video ngoài YouTube; và bổ sung các tính năng tương tác như biểu tượng cảm xúc thời gian thực, chia sẻ màn hình giữa các thành viên trong phòng. Đây sẽ là nền tảng để phát triển sản phẩm thành một ứng dụng hoàn chỉnh, sẵn sàng triển khai ở quy mô lớn hơn.

TÀI LIỆU THAM KHẢO

1. IETF (2011), *RFC 6455: The WebSocket Protocol*, Internet Engineering Task Force.
2. IETF (2015), *RFC 7519: JSON Web Token (JWT)*, Internet Engineering Task Force.
3. Socket.IO Documentation, *Real-time bidirectional event-based communication*, URL: <https://socket.io/docs/v4/>.
4. YouTube Developers, *YouTube Iframe Player API Reference*, URL: https://developers.google.com/youtube/iframe_api_reference.
5. PostgreSQL Global Development Group, *PostgreSQL 16 Documentation*, URL: <https://www.postgresql.org/docs/>.
6. Redis Ltd, *Redis Documentation and Commands*, URL: <https://redis.io/docs/>.
7. Node.js Foundation, *Node.js v20 LTS Documentation*, URL: <https://nodejs.org/docs/>.

PHỤ LỤC

Phụ lục A: Cấu trúc thư mục mã nguồn

- **Backend:** `src/config` (Postgres, Redis), `src/controllers`, `src/routes`, `src/services`, `src/sockets`, `src/db.ts`, `src/index.ts`.
- **Frontend:** `src/api`, `src/components`, `src/pages`, `src/main.ts`, `src/style.css`, `index.html`.

Phụ lục B: Cấu hình môi trường và Cài đặt

1. Khởi chạy Postgres & Redis: `docker compose up -d`
2. Khởi chạy Backend Server: `cd backend && npm install && npm run dev`
3. Khởi chạy Frontend Client: `cd frontend && npm install && npm run dev`

Phụ lục C: Link Repository và Video Demo

- **GitHub Repository:** <https://github.com/namxivin/CWH>
- **Video Demo:** https://youtube.com/watch?v=demo_hus